FilsPay Handover

Submission document · Team LGTM

Team

Member	Owned
Akheel Nazim	Auditing, finding the repo's planted traps, designing the AI workflow, fixes
Ahmed Khalaf	Fixes, deployment, load testing, UI fixes

Live URLs

What	URL
Checkout	https://merchant.139-59-92-10.sslip.io/pay/<link token>
Merchant dashboard	https://merchant.139-59-92-10.sslip.io/
Admin console	https://admin.139-59-92-10.sslip.io/
Public API	https://api.139-59-92-10.sslip.io/ (e.g. /healthz, /v1/…)

Repository

What	Value
Public fork	https://github.com/akheelnazim/fils-pay
Last commit before the deadline	[commit ID]

Contents

1. Summary 3
2. Architecture and deployment 4
3. Capacity 6
4. Findings report 7
5. UI and UX changes 29
6. AI usage 30
Appendix 33

1. Summary

What we found. We audited the FilsPay handover with AI and checked every claim in the code. We confirmed 109 findings: 88 fixed, 2 partly fixed (M19, S19), and 19 recorded but not fixed. By class: security holes (19), money-out races and duplicates (9), money maths that lost or invented fils (21), settlement timing (5), scripts and UI (17), deploy blockers (6), test-suite faults (3), 22 gaps in the half-built refund feature, and 7 dependency and performance items.

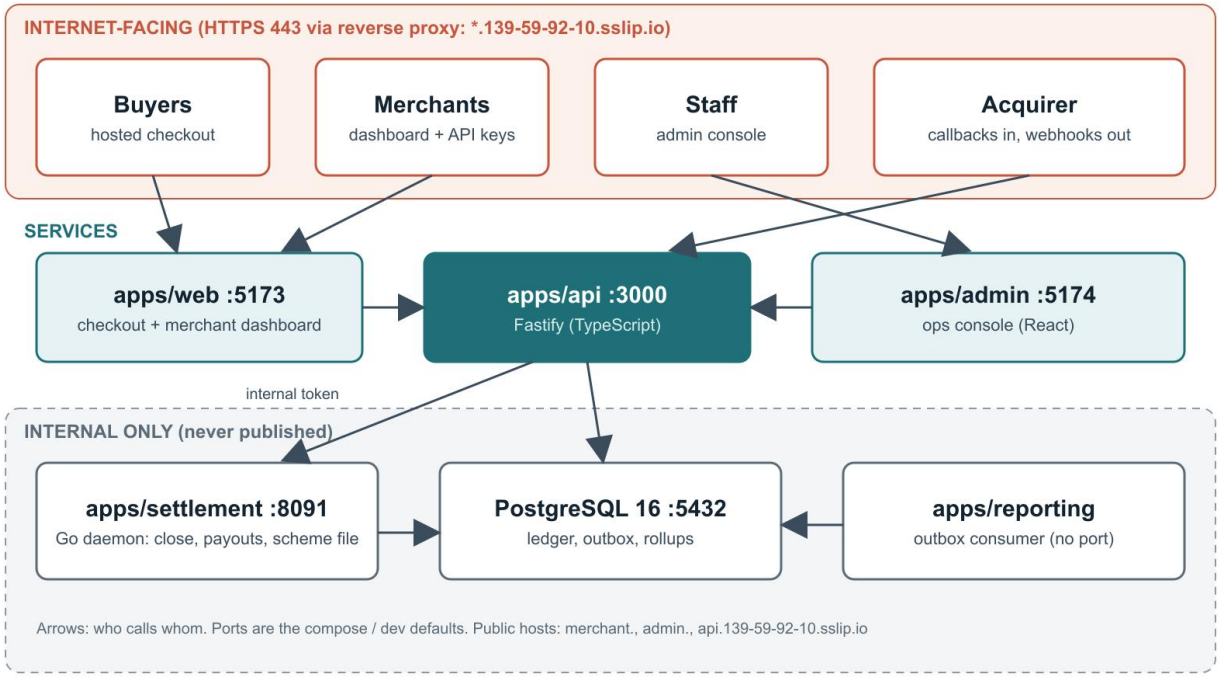
What we fixed. Every security, race, money and timing bug from the audit, plus the ones our reviewer agents and later scans found. Each fix has its own failing test written first, a reviewer agent's sign-off, an entry in `fix-docs/FIXLOG.md` and one commit on `main`. We finished FP-300 (refunds): both refund addresses work, refunds are capped at what was captured, safe under concurrency, reverse the right fee, reach the webhook, reduce the next payout, show in reports, and keep reconciliation at a zero gap. An end-to-end test proves it.

What we left alone. Anything that needs production data or a Finance decision: the March sweep's production postings (X14), correcting historical payout estimates (M19), journaling past scheme residue (M14), and rotating the leaked acquirer secret (S11). We never ran a `scripts/` one-off against data. We kept every ADR's decision, even where the audit proposed breaking one (M9). A final scan found 22 more issues; 4 are fixed (S13, M21, T5 and part of S19) and 18 are recorded with fixes proposed, two of them high-severity security (S14, S15).

Main root cause. FilsPay's safeguards lived in documents, comments and postmortems instead of in the code and the database. The ADRs required integer fils, but TypeScript and Go each rewrote money maths with floats and cents. Postmortems marked unique indexes and locks as "done" that were never built. Tenant checks sat on the route but not on the query. The repo's own AI instruction files were planted with false rules. Most findings trace back to a rule that was written down but never enforced.

2. Architecture and deployment

Services and ports



Service	Port	Faces the internet?	Notes
apps/web (checkout + merchant dashboard)	5173 (dev); merchant.139-59-92-10.sslip.io	Yes, through the proxy	Static React build
apps/admin (ops console)	5174 (dev); admin.139-59-92-10.sslip.io	Yes, through the proxy	Staff only
apps/api (Fastify)	3000; api.139-59-92-10.sslip.io	Yes, through the proxy	`/healthz`, `/readyz`
apps/settlement (Go daemon)	8091 internal API	No	Called by the API with an internal token
apps/reporting (worker)	none	No	Reads the outbox
PostgreSQL 16	5432	No (must not be published)	One database

Redeploy

The stack is defined in `ops/docker-compose.yml`, with Dockerfiles we added in fix D5. Locally it runs with:

```
docker compose -f ops/docker-compose.yml up -d --build
docker compose -f ops/docker-compose.yml run --rm migrate
```

[The one command used on the deployment server, e.g. a deploy script or CI job, and where it lives in the repo]

Where secrets come from

- Secrets are no longer committed (fix S11): `ops/.env.docker` is ignored, and `ops/.env.docker.example` lists every setting with placeholders.
- Required in production: `WEBHOOK_SECRET_ENCRYPTION_KEY` (32 bytes), `ACQUIRER_CALLBACK_SIGNING_SECRET`, `SETTLEMENT_INTERNAL_TOKEN`, `DATABASE_URL`, `CORS_ORIGINS`.

- On the server they come from an env file with restricted permissions, outside the repo.
- Known gap: compose has no `env_file`, so `.env.docker` is not loaded automatically (noted under D4 and scan item S15).

Health checks and restart policies

- Postgres: `pg_isready` health check in compose; every service waits for it.
- API: `GET /healthz` (liveness, no database) and `GET /readyz` (readiness).
- **[Restart policy per service on the server, e.g. restart: unless-stopped]**
- **[Health check wired into the proxy / platform]**

Redeploying without interrupting a settlement run

- The daemon stops cleanly on SIGTERM (`signal.NotifyContext` in `cmd/settlement/main.go`), and its internal server allows a long shutdown grace so a batch close isn't cut off.
- A batch close is one database transaction: if it is stopped, it rolls back. Re-closing a closed batch is a no-op (F8), and `settlement.paid` events have a unique index (R3), so a rerun after a redeploy can't double-pay or double-notify.
- Run exactly one settlement daemon. The close is due at 00:15 Asia/Bahrain (fixed in T1), so avoid deploying between 00:00 and 00:30 Bahrain time.
- **[Deployment procedure actually used: e.g. redeploy api/web/admin first, then settlement outside the close window; how you checked no close was running]**

3. Capacity

Not measured yet. The sections below are placeholders to fill after the load test.

Estimate on the fixed scenario

[Estimated simulated users served, and how the estimate was made]

Our load test

Item	Value
Tool	[e.g. k6 / Artillery / autocannon]
Method	[scenario, ramp, duration, data set]
Results	[requests/s, p50/p95/p99 latency, error rate at each step]
Estimated vs measured	[estimate] vs [measured]

[Load test graphs (also in the appendix)]

Where the bottleneck is

What the code already tells us, to check against the measurements:

- The API runs in a 512 MB container. The transactions export now streams (X3), so one big export can no longer exhaust it.
- Missing indexes on `ledger_entries.transaction_id`, `.settlement_batch_id` and `transaction_events.settlement_batch_id` (P2) mean sequential scans on refund and reconciliation queries.
- Balance rebuild runs four queries per merchant across 18k merchants (P3); status recompute and scheme export write row by row (P4).
- All services share one database role and one connection pool budget.

[Measured bottleneck]

Turning away excess load and recovering

Today nothing rate-limits requests (scan items S16, S17), so excess load queues instead of being refused.

[What the deployed stack does: proxy rate limits / connection limits / 429 or 503 responses, and how it recovered after the test]

4. Findings report

Every finding below was confirmed in the code before it was fixed. Each fixed one has a failing test written first (`*.fix.test.ts` or `*_fix_test.go`, named after its ID), a reviewer agent's check, and one commit on `main` named after the ID (`fix(S1): …`, `feat(FP-300/F3): …`). Full entries: `fix-docs/FIXLOG.md`. Sources: the audit (`fix-docs/filspay-audit-findings.md`), the evidence sheets, reviewer agents, two overall reviews and a final scan (`fix-docs/scan-2026-10-30.md`).

Money is in fils (1 BD = 1000 fils).

4.1 Security (S)

S1 Login works without the OTP

Symptom	The cookie issued after the password step works on every merchant route.
How found / reproduced	Audit; reproduced on the running API with a pending session.
Root cause	lookupSession reads the live_sessions view, which hides revoked/expired sessions but not unverified ones.
Fix	Add AND s.authenticated = true to lookupSession.
Verified by	auth/sessions.fix.test.ts: pending session returned a row before, null now.
Effect on existing data	None: code-only change.

S2 OTP and session issued to another merchant's user

Symptom	Logging in to merchant B can open merchant A's account when both share an email.
How found / reproduced	Audit; reproduced on the running API (Beta login landed in Alpha's data).
Root cause	Login looked the user up again by email across all merchants (LIMIT 1) instead of using the user it had just verified.
Fix	Use the verified user as the OTP recipient; delete the unused lookup.
Verified by	routes/auth.fix.test.ts: session went to A's user before, B's now.
Effect on existing data	None: code-only change.

S3 Cancel another merchant's payment link (IDOR)

Symptom	Any merchant can cancel any other merchant's link by id.
How found / reproduced	Audit + evidence clue.
Root cause	Route checks the URL merchant; cancelPaymentLink updates by link id alone.
Fix	Pass merchant id; both queries add AND merchant_id = $2 (foreign link answers 404).
Verified by	routes/paymentLinks.fix.test.ts: 204 before, 404 now.
Effect on existing data	None: code-only change.

S4 Expired payment link still payable (FP-187)

Symptom	Link shows expired but payment goes through.
How found / reproduced	Audit + evidence clue.
Root cause	Expiry judged against the browser-sent submittedAt.

Fix	Expiry uses the server clock; submittedAt ignored. Proper fix: drop the field from the API and checkout.
Verified by	paymentLinks/pay.fix.test.ts: 200 before, 410 now.
Effect on existing data	None: code-only change.

S5 Outbound webhook signatures can't be verified (FP-240)

Symptom	Merchants reject every webhook.
How found / reproduced	Audit + evidence clue.
Root cause	Signer signs sorted-key JSON; dispatcher sends JSON.stringify (different bytes).
Fix	Dispatcher and test-send send canonicalJson(payload), the exact signed string.
Verified by	webhooks/sign.fix.test.ts: raw-bytes HMAC failed before, passes now.
Effect on existing data	None: code-only change.

S6 Inbound acquirer signature check weak (FP-240)

Symptom	Unsigned or replayed callbacks accepted.
How found / reproduced	Audit.
Root cause	Signature optional by default, no timestamp window, non-constant-time compare.
Fix	Required by default, 300 s tolerance, timingSafeEqualHex. Proper fix: verify raw bytes.
Verified by	webhooks/inboundVerify.fix.test.ts; two tests that locked in the old behaviour updated.
Effect on existing data	None: code-only change.

S7 Rotated API key keeps working

Symptom	Old key still authenticates after rotation.
How found / reproduced	Audit.
Root cause	Rotation issued a new key but never set revoked_at on the old one.
Fix	Revoke the old key in the same transaction.
Verified by	admin/apiKeys.fix.test.ts; admin.test.ts asserted the bug and was corrected.
Effect on existing data	Keys rotated before the fix stay active until revoked by hand.

S8 Impersonation gets write access

Symptom	Staff 'view as merchant' can change merchant data, logged as the merchant.
How found / reproduced	Audit; reviewer found role-free write routes the audit missed.
Root cause	super_admin mapped to owner; capture/void/saved filters had no role check.
Fix	All admin roles map to support; impersonated non-GET requests get 403.
Verified by	admin/impersonation.fix.test.ts.
Effect on existing data	None: code-only change.

S9 Weak, brute-forceable OTP

Symptom	20 of 20 parallel guesses checked against a limit of 5.
How found / reproduced	Audit; reviewer found resend reset the attempt budget.
Root cause	Math.random codes; check-then-increment attempt counter.
Fix	crypto.randomInt; atomic UPDATE ... RETURNING claims an attempt; resend keeps the count.
Verified by	auth/otp.fix.test.ts.
Effect on existing data	None: code-only change.

S10 Credit self-approval compares display names

Symptom	One operator can approve their own credit after a rename.
How found / reproduced	Audit.
Root cause	assertDistinctApprover compared displayName.
Fix	Compare adminUserId.
Verified by	admin/creditApproval.fix.test.ts.
Effect on existing data	None: code-only change.

S11 Acquirer secret committed to git

Symptom	Signing secret in ops/.env.docker.
How found / reproduced	Audit.
Root cause	.gitignore covered only .env.
Fix	Untracked file, ignore .env.*, added .env.docker.example.
Verified by	scripts/committed_secrets.fix.test.ts.
Effect on existing data	Secret still in git history and must be rotated with the acquirer (not done: needs the acquirer and a force-push).

S12 Any admin can issue merchant credits

Symptom	Support login could post credits up to 50.000 BD.
How found / reproduced	Reviewer agent while fixing S8/S10.
Root cause	Create route only required an admin session.
Fix	requireAdminRole('finance','super_admin').
Verified by	routes/adminCredits.fix.test.ts: 201 before, 403 now.
Effect on existing data	None: code-only change.

S13 Any merchant can read another merchant's transactions

Symptom	GET /v1/reporting/transactions with another merchant's id in the filter returns up to 500 of their transactions.
How found / reproduced	Final code scan (agent read the code, main session re-read it).
Root cause	resolveReportMerchantId accepted any merchant that is a split child of any parent, without checking the caller is that parent.

Fix	Require the split relationship's parent to be the authenticated caller.
Verified by	reporting/scope.s13.fix.test.ts (self, own child, another parent's child, unrelated merchant); another parent's child was returned before.
Effect on existing data	None: code-only change.

S19 Reporting summary unauthenticated; disabled admins keep impersonation (partly fixed)

Symptom	export-summary answered without a login (500); a disabled admin's impersonation grant kept working until expiry.
How found / reproduced	Final code scan.
Root cause	The route read merchant auth without requiring it; grant lookup didn't check the admin was still active.
Fix	Require merchant authentication; require an active admin on every grant lookup. Still open: rotating the session after OTP.
Verified by	reporting.s19.fix.test.ts (500 !== 401 before), impersonation.s19.fix.test.ts.
Effect on existing data	None: code-only change.

4.2 Races and duplicates (R)

R1 Payment link charged twice (incident 2026-05-14)

Symptom	Two buyers charged for one link; a buyer told 'paid' for someone else's transaction.
How found / reproduced	Audit; postmortem's conclusion contradicted by code.
Root cause	Expired reservation can be re-claimed; finalize/release keyed only on status; a finalize miss still returned 200.
Fix	Reservation token; takeover voids the authorization and answers 409; finalize miss answers 409 and logs for refund.
Verified by	paymentLinks/pay.r1.fix.test.ts (three failed before).
Effect on existing data	None: code-only change.

R2 Duplicate acquirer callback notes (FP-201)

Symptom	Same callback saved twice.
How found / reproduced	Audit; claimed 'processed_events' fix never existed.
Root cause	Plain INSERT, no unique constraint.
Fix	Migration 0079 archives duplicates and adds a unique index; ON CONFLICT DO NOTHING.
Verified by	acquirer/callbacks.fix.test.ts (2 and 5 notes before, 1 now).
Effect on existing data	Duplicate notes moved to an archive table (reversible).

R3 Duplicate settlement.paid notifications (2026-08-21)

Symptom	Merchant gets two payout webhooks.
How found / reproduced	Audit; FP-208 advisory lock never existed.
Root cause	NOT EXISTS check then INSERT, no unique index.

Fix	Migration 0078 archives duplicates, adds a partial unique index; ON CONFLICT DO NOTHING.
Verified by	payout_notify_fix_test.go (concurrent scans, 20 repetitions).
Effect on existing data	44 duplicate event groups archived locally (reversible).

R5 Adjustment CSV importable twice

Symptom	Re-upload posts every row again.
How found / reproduced	Audit.
Root cause	Nothing identified an import.
Fix	Migration 0075 adds a unique content hash; repeat answers 409.
Verified by	admin/adjustmentImport.fix.test.ts.
Effect on existing data	Earlier imports keep a NULL hash.

R6 Reservation can expire during a slow capture

Symptom	Second buyer can pay while the first capture is in flight.
How found / reproduced	Found after R1 (follow-up review).
Root cause	Reservation timeout shorter than a slow capture.
Fix	Migration 0081 marks the capture on the link; a marked reservation can't be reclaimed.
Verified by	pay.r6.fix.test.ts, checkoutError.r6.fix.test.ts.
Effect on existing data	Links reserved before 0081 handled by R9.

R7 Stuck payment links had no recovery

Symptom	Uncertain captures leave links stuck processing.
How found / reproduced	Follow-up from R6.
Root cause	No operator workflow.
Fix	Audited finance-only recover-capture route; books must agree; 15-minute minimum age.
Verified by	recover.r7.fix.test.ts.
Effect on existing data	None: code-only change.

R8 Recovery can't ask the acquirer

Symptom	Every stuck link needed a manual portal check.
How found / reproduced	R7's proper fix.
Root cause	Acquirer interface had no lookup.
Fix	Optional lookupCapture; recovery asks first, falls back to the operator.
Verified by	recover.r8.fix.test.ts.
Effect on existing data	Simulator lookups are in-memory (lost on restart).

R9 Stuck links can't be listed; pre-0081 links unrecoverable

Symptom	No list; old links answer not_awaiting_recovery.

How found / reproduced	Follow-up review.
Root cause	Recovery found the transaction only through the marker.
Fix	GET .../payment-links/stuck; recovery accepts a checked transactionId.
Verified by	recover.r9.fix.test.ts.
Effect on existing data	None: code-only change.

4.3 Money correctness (M)

M1 CSV import reads BD as cents (10x)

Symptom	4.75 BD posted as 0.475 BD.
How found / reproduced	Audit; reproduced by calling the parser.
Root cause	2-decimal cents parser instead of parseBhdToFils (ADR-0001).
Fix	parseBhdToFils; cents helper deleted; tests asserting cents corrected.
Verified by	admin/adjustmentCsvParsing.fix.test.ts.
Effect on existing data	Past imports posted at a tenth; need a Finance review (not done).

M2 Scheme statement always a few fils high

Symptom	Statement over by up to 9 fils per line.
How found / reproduced	Audit + evidence clue; audit's own fix still used a float, corrected.
Root cause	Untruncated gross float rounded half-up into NUMERIC(10,2).
Fix	Store the truncated amount as an exact string; float helper deleted.
Verified by	settlement/batch_fix_test.go.
Effect on existing data	Rows already exported keep their values (files already sent).

M3 Split payouts short every batch

Symptom	Children's payouts don't add up to the parent's net.
How found / reproduced	Audit + evidence clue; reviewer found a flaw in the first fix.
Root cause	Each share rounded on its own; remainder never paid.
Fix	Largest-remainder allocation; shares never negative, total exact.
Verified by	splits/shares_fix_test.go, allocate_fix_test.go.
Effect on existing data	Past remainders sit in parents' payables.

M4 Net payout computed in float32

Symptom	Bigger batches paid a few fils off.
How found / reproduced	Audit.
Root cause	float32 has ~7 significant digits.
Fix	return grossFils - feeFils.
Verified by	settlement/payable_fix_test.go.

Effect on existing data	None: code-only change.

M5 Lifetime payout estimate drifts

Symptom	Single fils vanish past 16,777.216 BD.
How found / reproduced	Audit.
Root cause	REAL column, float32 sums.
Fix	Migration 0074 to BIGINT; int64 in Go.
Verified by	payout_estimate_fix_test.go.
Effect on existing data	Old drift kept; see M19.

M6 Platform volume counter overflows int32

Symptom	Counter wraps negative; 15 bps tier discount switches off.
How found / reproduced	Audit.
Root cause	INTEGER column, int32 in Go; tier starts 147 M fils below the max.
Fix	Migration 0073 to BIGINT; int64.
Verified by	platform_volume_fix_test.go, platformVolumeTier.fix.test.ts.
Effect on existing data	Widened in place; no values lost.

M7 Fee shown 245 bps, charged 225 bps

Symptom	Staff console shows the wrong rate.
How found / reproduced	Audit + evidence clue.
Root cause	Display ordered by created_at; billing by effective_from.
Fix	Display uses billing's rule; Go tie-break added.
Verified by	currentFeeScheduleDisplay.fix.test.ts, fee_schedule_fix_test.go.
Effect on existing data	None: code-only change.

M8 MDR recalculation rewrites paid batches

Symptom	Paid batch figures change after a rate update.
How found / reproduced	Audit + evidence clue.
Root cause	status <> 'open' includes paid.
Fix	Skip batch and cache updates for paid batches; ledger adjustment kept; no re-posting.
Verified by	recalculate_mdr_fix_test.go.
Effect on existing data	Batches already rewritten keep the rewritten figures.

M9 'Rebuild cached balances' gives a wrong figure (FP-203)

Symptom	Alpha Retail 135K to 7K after one click.
How found / reproduced	Audit; audit's fix rejected because it broke ADR-0004.

Root cause	View summed gross export lines over every batch, paid ones included.
Fix	Rebuild from net of closed batches (still settled data, per ADR-0004).
Verified by	ledger/refresh_fix_test.go (five failed before).
Effect on existing data	Next rebuild corrects the cache; ADR-0006 adjustments still reset by design.

M10 Settlement summary double-counts multi-capture

Symptom	Delta Wholesale ~38% over.
How found / reproduced	Audit; perf doc's 'timing difference' was double counting.
Root cause	Running total summed once per capture event.
Fix	Sum te.amount_fils.
Verified by	settlement/summary.fix.test.ts.
Effect on existing data	None: code-only change.

M11 Reconciliation pending uses the base rate

Symptom	Unexplained gap for re-priced merchants.
How found / reproduced	Audit; the '0046 trigger' never existed.
Root cause	Used merchants.mdr_bps.
Fix	Use the fee each capture recorded, else the schedule at that time.
Verified by	reconciliationPending.fix.test.ts.
Effect on existing data	None: code-only change.

M12 Unused discount parser reads BD as cents

Symptom	None yet (no caller).
How found / reproduced	Audit.
Root cause	parseFloat * 100.
Fix	Deleted.
Verified by	discountAdjustment.fix.test.ts.
Effect on existing data	None: code-only change.

M13 Reconciliation pending fee uses float rates

Symptom	1 fil off per capture on .5 boundaries.
How found / reproduced	ADR-0001 sweep.
Root cause	Float maths, rounds half-up.
Fix	Integer applyRateFils, same steps as capture.
Verified by	fees/estimate.fix.test.ts.
Effect on existing data	None: code-only change.

M14 Scheme rounding residue never journaled

Symptom	Ledger, report and scheme statement can't agree.
How found / reproduced	ADR-0002 check (ADR said 'implemented').
Root cause	No code posted residue.
Fix	CloseBatch posts a balanced residue group per line.
Verified by	settlement/residue_fix_test.go.
Effect on existing data	Batches closed April to now lack residue; catch-up needs Finance (not run).

M15 Balance rebuild can overwrite a concurrent batch close

Symptom	Cache misses a batch that closed during rebuild.
How found / reproduced	Developer TODO in code, confirmed.
Root cause	Read-then-upsert with no lock.
Fix	Lock the cache row FOR UPDATE before reading the ledger.
Verified by	ledger/rebuild_fix_test.go.
Effect on existing data	None: code-only change.

M16 Scheme export stores a decimal amount

Symptom	Decimal money column, against ADR-0001.
How found / reproduced	ADR-0001 sweep.
Root cause	Migration 0050.
Fix	Migration 0080 backfills integer exported_amount_fils and drops the decimal.
Verified by	scheme_export_m16_fix_test.go.
Effect on existing data	Backfilled; historical re-exports preserved.

M17 Go rate and split maths can overflow

Symptom	Huge amount returned a negative fee.
How found / reproduced	Follow-up review.
Root cause	int64 multiply before divide.
Fix	Exact intermediates; overflow returns an error and rolls back.
Verified by	rate_m17, shares_m17, discounted_fee_m17 tests.
Effect on existing data	None: code-only change.

M18 Split payouts skip the lifetime estimate

Symptom	Estimate omits split payouts.
How found / reproduced	Follow-up review.
Root cause	Split path never updated it.
Fix	Increment by the amount allocated.
Verified by	payout_estimate_m18_fix_test.go.

Effect on existing data	Past split payouts missing; see M19.

M19 Historical payout estimates need reconciliation (partly fixed)

Symptom	Stored estimates differ from what was paid.
How found / reproduced	Follow-up after M5/M18.
Root cause	Old float drift and missing split payouts.
Fix	Read-only drift report added; correction waits for Finance.
Verified by	payout_estimate_m19_fix_test.go.
Effect on existing data	Not overwritten on purpose: production totals need Finance sign-off.

M20 Monthly estimate overstates partial splits

Symptom	Estimate above what was paid.
How found / reproduced	Follow-up review.
Root cause	Summed full batch net.
Fix	Sum recognized payout ledger credits.
Verified by	payout_estimate_m20_fix_test.go.
Effect on existing data	None: code-only change.

M21 TypeScript and Go apply the discount in a different order

Symptom	Settlement fee 1 fil different from the capture ledger.
How found / reproduced	Final code scan (copy-paste drift between TS and Go).
Root cause	Settlement discounted the amount before rounding MDR; capture discounts the rounded fee.
Fix	MDR first, then the discount on the rounded fee, with checked integer maths.
Verified by	discounted_fee_m21_fix_test.go (10,200 at 250 bps, 10% discount: got 230, want 229); old test that asserted the old order updated.
Effect on existing data	Closed batches are not recalculated.

4.4 Settlement timing and test merchants (T)

T1 Daily close runs at 03:00 Bahrain, not midnight

Symptom	Every close and payout three hours late.
How found / reproduced	Audit; verified with a script.
Root cause	Daemon passed UTC time to a function that uses the argument's zone.
Fix	Convert to Asia/Bahrain before BusinessDayWindow.
Verified by	scheduler/daemon_fix_test.go.
Effect on existing data	None: code-only change.

T2 Admin tool moves test merchants into real settlement

Symptom	Sandbox captures paid out as real money.

How found / reproduced	Audit.
Root cause	Recompute cleared test_mode when it saw no open authorizations.
Fix	Removed the flag change.
Verified by	adminMerchantsRecompute.fix.test.ts.
Effect on existing data	Merchants flipped earlier need checking by hand.

T3 Recompute always reports 0 open authorizations

Symptom	Wrong count in the console.
How found / reproduced	Found while fixing T2.
Root cause	Counted inside a query that excludes authorized rows.
Fix	Separate count query.
Verified by	transactionStatusRecompute.fix.test.ts.
Effect on existing data	None: code-only change.

T4 Bad polling interval crashes the daemon

Symptom	Daemon panics at start.
How found / reproduced	Follow-up review.
Root cause	No range check before time.NewTicker.
Fix	Reject out-of-range intervals at config load.
Verified by	config/poll_interval_fix_test.go.
Effect on existing data	None: code-only change.

T5 Admin settlement page pre-fills the wrong (UTC) date

Symptom	Form defaults to the wrong business day around midnight.
How found / reproduced	Final code scan.
Root cause	Browser-local date arithmetic combined with UTC formatting.
Fix	Derive Bahrain's calendar date and subtract one day, whatever the browser's time zone.
Verified by	SettlementPage.t5.fix.test.ts (Bahrain midnight, leap day, year end).
Effect on existing data	None: code-only change.

4.5 Scripts and other (X)

X1 Hold clean-up can delete every hold

Symptom	Quiet night runs DELETE FROM holds.
How found / reproduced	Audit.
Root cause	Filter helper drops an empty filter.
Fix	One UPDATE ... SET status='expired'.
Verified by	scripts/expire_holds.fix.test.ts.

Effect on existing data	Holds already deleted can't be restored from code.

X2 March sweep script not idempotent

Symptom	Each run posts the residue again; 'dry run' posted.
How found / reproduced	Audit + evidence clue.
Root cause	Fresh UUID per run; no guard; no dry-run flag.
Fix	Advisory lock + refuse if posted; real --dry-run.
Verified by	scripts/reconcile_march.fix.test.ts.
Effect on existing data	Production run count unknown: X14.

X3 Large export can run the API out of memory

Symptom	Whole history built in a 512 MB process.
How found / reproduced	Audit.
Root cause	In-memory builder; streaming version unused and buggy.
Fix	Keyset-paged async stream.
Verified by	transactions/exportCsv.fix.test.ts.
Effect on existing data	None: code-only change.

X4 Credit threshold can be split around

Symptom	2 x 30 BD credit posts without a second approver.
How found / reproduced	Audit.
Root cause	Checked base only, one request at a time.
Fix	Base + reversal + recent credits, merchant row locked.
Verified by	admin/credits.x4.fix.test.ts.
Effect on existing data	None: code-only change.

X5 March sweep charges merchants the residue

Symptom	Merchant payables reduced.
How found / reproduced	ADR-0002 check.
Root cause	Legs reversed vs ADR-0002.
Fix	Clearing credit / residue debit.
Verified by	reconcile_march.x5.fix.test.ts.
Effect on existing data	Production postings need reversal with Finance (X14).

X6 One bad event stops reporting for good

Symptom	Dashboards freeze.
How found / reproduced	Surfaced by the baseline test failures.

Root cause	Whole pass in one transaction.
Fix	Savepoint per event; bad ones go to a dead-letter table (0076).
Verified by	runConsumerPass.x6.fix.test.ts.
Effect on existing data	None: code-only change.

X8 Reporting crashes on its own lag check

Symptom	Worker stops when a rollup row is missing.
How found / reproduced	Exposed after Q2.
Root cause	Assumed the row exists.
Fix	Treat missing as 0 and warn.
Verified by	applyEvent.fix.test.ts.
Effect on existing data	None: code-only change.

X9 Reconciliation page crashes on a negative gap

Symptom	Page throws on render.
How found / reproduced	Code comment lead.
Root cause	Display formatter rejected negatives.
Fix	Allow signed fils in display.
Verified by	web format.x9.fix.test.ts.
Effect on existing data	None: code-only change.

X10 Dashboard tab titled 'FilsPay Checkout'

Symptom	Wrong page identity.
How found / reproduced	Code comment lead.
Root cause	Hard-coded title.
Fix	Dashboard title; checkout sets its own.
Verified by	documentTitle.x10.fix.test.ts.
Effect on existing data	None: code-only change.

X11 Merchant UI root URL shows a 404 error screen

Symptom	Developer error page at /.
How found / reproduced	Running the app.
Root cause	No root route.
Fix	Redirect / to /dashboard.
Verified by	router.x11.fix.test.ts.
Effect on existing data	None: code-only change.

X12 Analytics accepts impossible dates

Symptom	2026-02-30 gives a 500.
How found / reproduced	Code comment lead.
Root cause	Date.parse normalises dates.
Fix	Exact calendar validation.
Verified by	analytics.x12.fix.test.ts.
Effect on existing data	None: code-only change.

X13 Malformed JSON returns 500

Symptom	Client error reported as server error.
How found / reproduced	Found while testing.
Root cause	Parser errors fell to the generic handler.
Fix	Map to 400 validation_error.
Verified by	errorHandler.x13.fix.test.ts.
Effect on existing data	None: code-only change.

X14 March sweep production postings (open)

Symptom	Unknown how many times it ran in production.
How found / reproduced	Evidence clue.
Root cause	Old script had no guard; log overwritten.
Fix	Not fixed: read-only reconciliation with Finance proposed.
Verified by	Not applicable (no production data touched).
Effect on existing data	Possible duplicate / wrong-direction entries remain.

4.6 Deploy and setup (D)

D1 Migration 0054 breaks a fresh database

Symptom	npm run migrate stops at 0054.
How found / reproduced	Fresh migrate.
Root cause	View on a table no migration creates; number reserved by ADR-0005.
Fix	Moved needed objects to idempotent 0077.
Verified by	db/migrate.fix.test.ts.
Effect on existing data	Old databases and fresh ones end the same.

D2 Migration 0055 uses non-existent columns

Symptom	Migrate stops at 0055.
How found / reproduced	Fresh migrate after D1.
Root cause	Wrong chart_of_accounts columns.

Fix	Dropped (accounts unused).
Verified by	db/migrate.d2.fix.test.ts.
Effect on existing data	None: code-only change.

D3 Dev webhook key is 34 bytes

Symptom	API refuses to boot in dev.
How found / reproduced	Local setup.
Root cause	Fallback string wrong length.
Fix	32-byte dev key.
Verified by	config.fix.test.ts.
Effect on existing data	None: code-only change.

D4 API sends no CORS headers

Symptom	Dashboard and console blocked by the browser.
How found / reproduced	Running the app.
Root cause	No CORS registered.
Fix	Small CORS plugin; CORS_ORIGINS setting.
Verified by	plugins/cors.fix.test.ts.
Effect on existing data	None: code-only change.

D5 Compose can't build or start

Symptom	Missing Dockerfiles; settlement exits.
How found / reproduced	docker compose up.
Root cause	Dockerfiles never committed; wrong command/env.
Fix	Node and Go Dockerfiles; daemon command and token.
Verified by	compose config, build, migrate, seed, all containers up.
Effect on existing data	None: code-only change.

4.7 FP-300 refunds (F)

F14 Every refund crashes on insert

Symptom	11 baseline refund tests fail.
How found / reproduced	Baseline test run.
Root cause	Explicit NULL reason vs CHECK constraint.
Fix	Default 'unspecified'.
Verified by	processRefund.f14.fix.test.ts.
Effect on existing data	None: code-only change.

F1 Refund more than was captured

Symptom	5,000 refunded on a 4,000 capture.
How found / reproduced	Audit.
Root cause	Used authorized, not captured.
Fix	captured - refunded.
Verified by	processRefund.f1.fix.test.ts.
Effect on existing data	None: code-only change.

F2 Refund another merchant's payment (IDOR)

Symptom	Merchant B refunds A's transaction.
How found / reproduced	Audit.
Root cause	Lookup by id only.
Fix	AND merchant_id = $2.
Verified by	initiateMerchantRefund.f2.fix.test.ts.
Effect on existing data	None: code-only change.

F3 Concurrent refunds both pass (FP-209)

Symptom	6,000 + 6,000 on 10,000 both succeed.
How found / reproduced	Audit; deterministic gated test.
Root cause	No lock; absolute overwrite.
Fix	FOR UPDATE and reserve before the acquirer call.
Verified by	processRefund.f3.fix.test.ts.
Effect on existing data	None: code-only change.

F4 Fee reversal ignores the discount

Symptom	255 reversed where 229 was kept.
How found / reproduced	Audit.
Root cause	Summed credits only.
Fix	Net credits minus debits.
Verified by	processRefund.f4.fix.test.ts.
Effect on existing data	None: code-only change.

F5 Partial fee reversals don't add up

Symptom	33 reversed of a 25 fee.
How found / reproduced	Audit.
Root cause	Each share rounded alone, no cap.
Fix	Cap at fee left; last refund takes the rest.
Verified by	processRefund.f5.fix.test.ts.

Effect on existing data	None: code-only change.

F6 Refund route was a 501 stub at the wrong address

Symptom	No refunds at all.
How found / reproduced	Audit; route decision by the team lead.
Root cause	Never wired; addresses disagreed.
Fix	Both /v1/refunds and /v1/merchants/:id/refunds, one handler; drawer restored. Includes F13.
Verified by	routes/refunds.fix.test.ts (43 cases on both addresses).
Effect on existing data	None: code-only change.

F7 refund.succeeded never sent to webhooks

Symptom	Integrations never hear of refunds.
How found / reproduced	Audit.
Root cause	No envelope case, not in the eligible list.
Fix	Envelope case first, then the list.
Verified by	envelope.f7.fix.test.ts.
Effect on existing data	No backlog flood: no older events existed.

F8 Settlement ignores refunds

Symptom	Paid 9,750 instead of 5,850.
How found / reproduced	Audit.
Root cause	No refund code in Go.
Fix	CloseBatch deducts refunds; too-big refunds roll to the next day.
Verified by	settlement/refund_fix_test.go.
Effect on existing data	None: code-only change.

F9 Refunds missing from the daily rollup

Symptom	Reports show no refunds; wrong day near midnight.
How found / reproduced	Audit; more missing than it said.
Root cause	No consumer case, no columns, UTC dates.
Fix	Migration 0082 columns; consumer and rebuild use Bahrain day.
Verified by	applyEvent.f9, reportingRebuild.f9 tests.
Effect on existing data	Columns default 0.

F15 Refund summary view always empty

Symptom	No rows ever.
How found / reproduced	Found while planning FP-300.

Root cause	Joined on the refund's own id.
Fix	Migration 0083 joins on transaction_id.
Verified by	refundSummary.f15.fix.test.ts.
Effect on existing data	View recreated.

F10 Reconciliation shows refunds as a gap

Symptom	gap -3,900 after a refund.
How found / reproduced	Audit.
Root cause	Pending ignored refunds.
Fix	Subtract unbatched refunds.
Verified by	reconciliationRefunds.fix.test.ts.
Effect on existing data	None: code-only change.

F11 Responses lack refundedFils

Symptom	Drawer offers the full amount again.
How found / reproduced	Audit.
Root cause	Field left out.
Fix	Added to API and web types.
Verified by	serialize.f11.fix.test.ts.
Effect on existing data	None: code-only change.

F12 Full refund of a partial capture never 'refunded'

Symptom	Stays partially_captured.
How found / reproduced	Audit.
Root cause	Capture check ran before refunds.
Fix	refunded >= captured gives refunded.
Verified by	refundState.f12.fix.test.ts.
Effect on existing data	None: code-only change.

F16-F18, F20 Fee recalculation vs refunded batches

Symptom	Net -50; fully refunded batch paid 50 fils.
How found / reproduced	Opus overall review.
Root cause	Recalculation re-rated refunded money.
Fix	Re-rate only the money a batch kept; cap the change.
Verified by	refund_fix_test.go TestF16/F18/F20.
Effect on existing data	None: code-only change.

F17, F19 Status and webhook count in-flight refunds

Symptom	False 'refunded' webhook.
How found / reproduced	Opus overall review.
Root cause	Used the reservation counter.
Fix	Use succeeded refund rows.
Verified by	processRefund.f17, transactionStatusRecompute.f19 tests.
Effect on existing data	None: code-only change.

F21 Reconciliation gap when a refund rolls to a later batch

Symptom	-3,900 / +3,900 on two days.
How found / reproduced	Opus overall review.
Root cause	Ledger and batch count the refund on different days.
Fix	Window-aware pending.
Verified by	reconciliationRefunds.f21.fix.test.ts.
Effect on existing data	None: code-only change.

F22 Pending refunds invisible

Symptom	Stuck reservations with no trace.
How found / reproduced	F3's proper fix.
Root cause	Row written only on success.
Fix	Migration 0084: pending status and pending_refund_fils.
Verified by	processRefund.f22.fix.test.ts.
Effect on existing data	None: code-only change.

4.8 Test suite (Q)

Q1 Flaky webhook backoff test

Symptom	Failed almost every run.
How found / reproduced	Baseline.
Root cause	Exact assert on a jittered value.
Fix	Fixed jitter in the test.
Verified by	backoff.fix.test.ts.
Effect on existing data	None: code-only change.

Q2 Archive test pollutes the shared DB

Symptom	4 reporting tests fail after API tests.
How found / reproduced	Baseline.
Root cause	Real event types with empty payloads.

Fix	Inert fixture type.
Verified by	Full suite run.
Effect on existing data	Local dev DB cleaned only.

Q3 Go packages deadlock in parallel

Symptom	deadlock detected.
How found / reproduced	Go suite.
Root cause	Packages share one DB.
Fix	go test -p 1.
Verified by	All Go packages pass.
Effect on existing data	None: code-only change.

4.9 Open findings from the final scan (not fixed)

A read-only scan after FP-300 found 22 issues. S13, M21, T5 and part of S19 are fixed (entries above); these 18 are still open. They are tracked in FIXLOG and described with evidence in `fix-docs/scan-2026-10-30.md`. X14 (above) is also open.

ID	Severity	Finding	Proposed fix
S14	High	SSRF through webhook URLs (no private-IP block, redirects followed, body echoed)	Block private ranges at create and send; no redirects
S15	High	Compose runs the API in development mode; no .dockerignore (secret and .git in image)	Production env, .dockerignore, non-root user
S16	Medium	OTP guess limit resets on every login; no rate limiting	Per-user and per-IP limits
S17	Medium	Public pay endpoint allows unlimited card testing	Rate limit and attempt cap per link
S18	Medium	Unsalted hash of card number, CVV and expiry stored	Stop storing CVV; keyed hash
R10	Medium	Reporting consumer can skip outbox events (cursor by id, not commit order)	Cursor on commit order
X15	Medium	Admin 'Rebuild analytics' double-counts unconsumed events	Rebuild up to the watermark
D6	Medium	Compose gives the API no settlement token	Set the token
X16	Low	Payout trends can stay 'closed' forever	Handle settlement.paid
X17	Low	Call to a Go endpoint that doesn't exist; docs name env vars nothing reads	Remove or implement
X18	Low	Dead code (v1/ tree and others)	Delete
V1	Medium	npm audit: 3 high, 5 moderate	Upgrade
V2	Medium	govulncheck: 37 reachable (Go 1.22 out of support)	Upgrade Go and modules
V3	Low	API image on Node 20 (EOL); build tools in runtime image	Node 22, slim runtime stage
P1	Low	Unused dependencies	Remove
P2	Medium	Missing indexes on ledger and event foreign keys	Add indexes
P3	Medium	Balance rebuild runs four queries per merchant (18k merchants)	Set-based rebuild
P4	Low	Row-by-row writes that could be one statement	Batch the writes

4.10 Root causes

Grouping the findings by the cause they share:

Root cause	Findings	The design decision behind it
Money rules on paper, not in code	M1, M2, M4, M5, M6, M12, M13, M16, M17, M21	ADR-0001 says integer fils, but money maths lives in two languages. The shared `packages/money` exists only for TypeScript; Go and several TS paths rewrote it with floats, float32, cents parsers, REAL/INTEGER columns and decimal copies. Nothing (lint, types or schema) stopped them.
Check-then-act with no database backstop	R1, R2, R3, R5, R6, S9, F3, X4, M15, X2	Uniqueness and limits were enforced by reading first and writing later. The postmortems claimed the backstops (a `processed_events` table, the FP-202 unique index, the FP-208 advisory lock) but none existed. Fix pattern: an atomic conditional `UPDATE … RETURNING` or `SELECT … FOR UPDATE`, plus a unique index.
Authorization checked at the route, not in the query	S1, S2, S3, S8, S12, S13, S19, F2	Routes checked the merchant in the URL or that an admin was logged in, while data functions took a bare id. Any id from another tenant then worked. Fix pattern: scope every query by the caller's merchant, and check roles server-side.
Trusting what comes from outside	S4, S5, S6, S7, S11, X12, X13 · open: S14, S17	Client clocks, unsigned callbacks, signatures over the wrong bytes, keys that outlive rotation, secrets in git. Security defaults were "optional".
Two copies of one rule drift apart	M3, M7, M10, M11, M14, M21, T1, T5, F9, F12, F15, X3, D4	Displays and reports re-derived figures instead of reading what billing recorded (fee shown vs charged, summary vs payouts). Docs and comments described behaviour the code didn't have: ADR-0002 marked "implemented" with no code, CHANGELOG saying the export streams, a comment saying CORS was locked down. UTC was used where the rule is the Bahrain day.
Operational tools that act without guards	X1, X2, X5, T2, M8, M9, X6, X8, D1, D2, D5	Scripts and console buttons wrote money data with no idempotency, no dry run, and no check of what state they were touching (paid batches, sandbox merchants, every hold).
A feature left half-built	F1–F22	Refunds had tables, helpers and a UI drawer, but the route returned 501, the insert always failed, and settlement, reporting, webhooks and reconciliation didn't know refunds existed. "Just wire it" would have shipped every gap above.

Across all of them: the repo was booby-trapped for AI. `CLAUDE.md`, `AGENTS.md` and `.cursorrules` said money was cents, settlement used UTC midnight, `Math.random` was fine for OTPs and refunds should stay 501. Inline comments ("claude says it's fine") were planted the same way. We deleted the three files first and treated ADRs, code and tests as the only truth.

4.11 Decisions and open risks

Decision / risk	Why	Safeguard we propose	Status
Rejected the audit's M9 fix (rebuild balances from the ledger)	ADR-0004 says the button rebuilds from settled data. The code follows the ADRs, not the other way round.	Fixed the settled-data path instead (net of closed batches), tested.	Closed
Moved migrations 0054–0056 to 0077	ADR-0005 reserves those numbers for settlement v2; 0054/0055 also broke a fresh migrate.	Idempotent 0077 so old and new databases end the same.	Closed
Kept `/v1/refunds` and added `/v1/merchants/:id/refunds`	Merchants already integrate on `/v1/refunds`; the AI planned to remove it (see section 6).	One handler; 43 tests run on both addresses.	Closed

Decision / risk	Why	Safeguard we propose	Status
Never ran `scripts/` one-offs against data	They post to the ledger; X2 shows a "dry run" that posted.	Real `--dry-run`, idempotency guard; Finance-led reconciliation.	X14 open
Did not overwrite historical payout estimates (M19)	Production totals used by ops and Finance; the ledger may be incomplete for old history.	Read-only drift report; Finance agrees figures, then an audited migration.	Open
No catch-up journal for April-to-now scheme residue (M14)	Changes closed batches' ledger; needs Finance.	One-off reviewed migration with Finance.	Open
Did not rotate the acquirer secret or rewrite git history (S11)	Needs the acquirer, and a force-push would break collaborators' clones.	Rotate with the acquirer before go-live; treat the old secret as public.	Open
Smallest fix over the proper fix	Brief: a small fix that works beats a rewrite. Bigger fixes are recorded per entry in FIXLOG.	e.g. S4 drop `submittedAt` from the API, S6 raw-body verify, T1 fix inside `BusinessDayWindow`.	Recorded
Refunds larger than the day's net roll to the next batch (F8)	A payout can never go negative and the debt is not lost.	Strict ordering would hold later refunds back; separate refund column on batches.	Closed
Final-scan findings not fixed	Found at the end; each needs the full test-first workflow.	Fix S14 (SSRF) and S15 (compose in dev mode) before real traffic.	Open (18)
Payment-link recovery lookups are in-memory (R8)	The acquirer simulator keeps no durable record.	A real acquirer inquiry API or a stored record.	Open
Refund summary view isn't refreshed automatically (F15)	Nothing in the code refreshes it, despite its comment.	Refresh from the reporting worker.	Open
Load test	[status at submission]	[ ]	[ ]

5. UI and UX changes

5.1 Refund drawer restored on the transaction page (F6, F11, F22)

Why it mattered: Merchants had no way to refund. The drawer now caps the amount at captured − refunded − pending and shows API errors.

Before	After
[screenshot]	[screenshot]

5.2 Reconciliation page no longer crashes on a negative gap (X9)

Why it mattered: A negative gap is valid; the page threw and showed nothing.

Before	After
[screenshot]	[screenshot]

5.3 Dashboard tab titled "FilsPay Dashboard" (X10)

Why it mattered: Every page was labelled "FilsPay Checkout" in tabs and history.

Before	After
[screenshot]	[screenshot]

5.4 Merchant UI root URL redirects to the dashboard (X11)

Why it mattered: Opening the base URL showed React Router's developer 404 screen.

Before	After
[screenshot]	[screenshot]

5.5 Checkout explains an uncertain or lost payment (R1, R6)

Why it mattered: A buyer who was charged but lost the reservation now sees "charged, contact the merchant" instead of a retry prompt that could charge them twice.

Before	After
[screenshot]	[screenshot]

5.6 Staff console shows the fee rate actually charged (M7)

Why it mattered: It showed 245 bps while 225 bps was billed.

Before	After
[screenshot]	[screenshot]

Recording: **[link to screen recording, if any]**

6. AI usage

Setup

One line: Claude Code (Opus) in the repo, driven by a custom fix-and-build skill, with cheaper subagents for tests (Haiku) and independent reviewers (Sonnet), and shared memory kept in git.

1. **Understand the codebase.** We first had Claude read the public GitHub repo. It found the repo was booby-trapped: `CLAUDE.md`, `AGENTS.md` and `.cursorrules` contained false rules (cents, UTC, "don't fix refunds"). We deleted all three in their own commit before any other work.
2. **Map it.** We built a floorplan (https://www.claudemap.org/en/explore/floorplan): an interactive map of every module, file and endpoint with plain-English descriptions and the calls between them. It lives in `floorplan/` and is queried with `node floorplan/query.mjs`.
3. **Audit.** Claude audited the code for security holes, money-out races and duplicates, money rounding and integer overflow, settlement timing and time zones, unsafe scripts, deploy blockers, broken ADR rules and UI errors. Each claim was then re-checked against the code and the ADRs before we trusted it ("Audit check" and "Against the ADRs" in FIXLOG).
4. **A workflow skill.** `.claude/skills/fix-and-build/SKILL.md` makes every fix follow the same steps: confirm the bug in code, check the ADRs (stop and ask on a conflict), have a smaller agent (Haiku) write a failing test with edge cases, apply the smallest fix and record the proper one, check every caller in TypeScript and Go, run the suites, have a Sonnet reviewer agent hunt for breakage, then write the FIXLOG entry and one descriptive commit.
5. **Commits and collaboration.** Claude committed and pushed on its own once a fix passed review: one branch per fix off a freshly pulled `main`, pull again, squash-merge as one commit. A colleague committed to `main` at the same time, so the skill says how to resolve and record conflicts (a `Conflicts:` line in the commit and FIXLOG).
6. **Shared memory in the repo.** Anything learned about the codebase went into `fix-docs/memory/` (commit rules, fix lifecycle, project decisions, pitfalls, which model does what) and `fix-docs/LESSONS.md`, so every session, ours and our colleague's, starts with the same knowledge.
7. **FP-300 refunds** were planned with the superpowers writing-plans skill (`docs/superpowers/plans/2026-10-30-fp-300-refunds.md`) and built one gap per commit. An Opus overall review at the end found F16–F21.

Where it lives: `.claude/skills/fix-and-build/`, `fix-docs/memory/`, `fix-docs/LESSONS.md`, `fix-docs/FIXLOG.md`, `floorplan/`, `docs/superpowers/plans/`, `CLAUDE.md` (rewritten by us).

Three annotated transcript excerpts

Quotes are taken from our Claude Code session transcripts (lightly trimmed with …). Our prompts are quoted as typed.

Excerpt 1: the AI was wrong. It planned to remove the refund address merchants already use

Finding it supports	F6 (refund route), and with it F13.
Prompt and context	Claude Code (Opus) on 30 Oct, using the superpowers writing-plans skill to plan FP-300. It had the repo, the audit (which said to register the route under `/v1/merchants`), the dashboard's RefundDrawer, and `docs/`.

> **Claude:** "Old `/v1/refunds` address: I plan to remove it. `docs/API.md` lists it, but the dashboard and the audit use the merchant-scoped address; I'll update API.md."
>
> **Us:** "dont remove the old refund addresss, wouldnt fixing it make more sense? you tell me."
>
> **Claude:** "you're right, fixing it makes more sense… `POST /v1/refunds` is the address the project docs give for merchants who connect with an API key, so removing it would break those integrations."

How we noticed	Merchants integrate with FilsPay over the API. Removing a documented endpoint to suit the dashboard would have broken every merchant already calling it. The AI followed the audit's suggestion literally and treated the docs as something to "update" rather than as a contract.
How we verified it	Checked the docs list POST `/v1/refunds` for API-key integrations. The route tests (`routes/refunds.fix.test.ts`, 43 cases) run every check against both addresses. We then asked Claude to go back over every earlier fix for anything else that changes what outside systems see. In the same answer it had called a UTC capture-date function a new bug; checking showed nothing calls it, so it was dropped, not "fixed".
What we changed	The plan and the code: both `/v1/refunds` (API keys) and `/v1/merchants/:merchantId/refunds` (dashboard) run one handler, the merchant always comes from the login, and the decision is recorded in FIXLOG (F6).

Excerpt 2: the audit's fix broke a binding rule (ADR-0004)

Finding it supports	M9 ("Rebuild cached balances" turned 135K into 7K), plus M2, X5 and D1/D2.
Prompt and context	Claude Code (Opus) on 29 Oct, with the audit, `docs/adr/` and the code. Prompt: "verify the audit findings against ADRs also".

> **Claude:** "The fixes it proposes follow them, with one exception… Correction to the audit: its fix for M2… still passes the amount as a decimal number. ADR-0001 forbids floating-point numbers on money… M9 needs your call (the audit's fix conflicts with ADR-0004), and migrations 0054–0056 break ADR-0005."
>
> **Us:** "the code should follow adr, not the other way around."
>
> **Claude:** "M9: the rebuild button keeps using settled data, as ADR-0004 says. The fix goes inside that path instead of switching to the ledger."

How we noticed	The audit was AI-written too, and its one-line M9 fix (swap to the ledger rebuild) would have quietly reversed an accepted architecture decision. The ADR cross-check also turned up a new bug, X5 (the March sweep charged merchants the residue).
How we verified it	`ledger/refresh_fix_test.go`: five cases failed before (gross instead of net, paid batches counted), all pass with the rebuild reading the net of closed batches. `db/migrate.fix.test.ts` and `migrate.d2.fix.test.ts` prove a fresh database migrates to the end with no 0054–0056 files.
What we changed	We made "ADRs are rules; a conflict means stop and ask" part of the skill and `CLAUDE.md`, and recorded the decisions in the audit file and FIXLOG ("Against the ADRs").

Excerpt 3: the reviewer agent caught the AI's own fix overpaying

Finding it supports	M3 (split payouts short every batch).
Prompt and context	The fix-and-build workflow: a Haiku subagent wrote the failing database test ("sum of settlement_batch_splits.allocated_fils = 10000, want 10001"), Opus wrote the fix, a read-only Sonnet reviewer checked it.

> **Reviewer (Sonnet):** "Problems found… the adjusted share can go negative… Counterexample: net=3, five children at 2000 bps… it pays 1+1+1+1 = 4 against a net of 3. That is an overpay, which is what M3 is meant to remove."
>
> **Claude:** "The reviewer found a real hole… Switching to the standard largest-remainder method… round every share down… hand out the leftover fils one at a time… Shares can't go negative and always add up exactly. I'm adding the reviewer's case to the test."

How we noticed	The first fix put the whole rounding difference on the largest share. The tests only had two or three children, so they passed; the reviewer built a case with five.
How we verified it	`splits/shares_fix_test.go` now includes the five-way 3-fil case; `allocate_fix_test.go` checks split rows, ledger debits and the parent's cache all equal the net. Full Go suite green; a re-review answered "OK to commit".

What we changed	The allocation algorithm (largest remainder, weights capped at 100%), and the lesson "reviewers find what the audit missed" in `LESSONS.md`.

Appendix

A. Tests that failed before any fix

Baseline on 29 Oct (TS suite on a fresh, seeded database): API 350/362, reporting 7/11. All 16 failures were explained and fixed:

Failures	Cause	Fixed by
11 refund tests	Every refund insert passed `reason` as NULL against a CHECK constraint	F14
1 dispatcher backoff test	Asserted an exact delay that has random jitter	Q1
4 reporting tests	Another test seeded real event types with empty payloads into the shared DB; one bad event then stopped the consumer for good	Q2, X6

B. Migrations we added

Migration	For
0073_platform_stats_bigint	M6
0074_lifetime_payout_estimate_bigint	M5
0075_adjustment_import_content_hash	R5
0076_reporting_dead_letter_events	X6
0077_views_and_revoke_after	D1 (replaces 0054/0056)
0078_settlement_paid_event_idempotency	R3
0079_acquirer_callback_note_idempotency	R2
0080_drop_scheme_export_decimal_amount	M16
0081_payment_link_capture_in_progress	R6
0082_merchant_daily_volume_refunds	F9
0083_refund_merchant_summary_join	F15
0084_refunds_pending	F22

C. FP-300 end-to-end test

`apps/api/src/refunds/refund.e2e.fix.test.ts`: authorize 10,000 fils → capture → refund 3,000 → refund the remaining 7,000 → a third refund answers 409. It checks every ledger group balances, `merchant_payable` and `platform_fee_revenue` net to 0, the full 250-fil fee is reversed, the status is `refunded`, two `refund.succeeded` webhooks are delivered with signatures that verify, and the reconciliation gap is 0. The payout reduction is tested on the Go side (`settlement/refund_fix_test.go`: capture 10,000 + refund 4,000 pays 5,850).

D. How to run the tests

```
docker start filspay-pg && npm run migrate
DATABASE_URL=postgres://filspay:filspay_dev_password@localhost:5432/filspay npm test
npx tsx --test scripts/*.test.ts
MSYS_NO_PATHCONV=1 docker run --rm -v "$PWD/apps/settlement:/src" -w /src \
  -e DATABASE_URL=postgres://filspay:filspay_dev_password@host.docker.internal:5432/filspay \
  golang:1.22 go test -p 1 ./...
```

E. Final test output

[Paste the final suite summary: TS pass/fail counts per workspace, scripts/, Go packages]

F. Load test graphs

[Graphs from section 3]

G. Commit message format

```
fix(<ID>): <what is fixed, in plain words>

What: <the bug>
Why: <root cause>
Impact: <who lost what, a fils example if money>
Fix: <the change>. Proper fix (if different): <…>
Test: <new test file>; failed before, passes now.
Checked: <callers/places reviewed>
```